

Program :-

```
for i in range(1,6):
    for j in range(1,6):
        if j <= 6-i:
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()
```

Output :-

```
* * * * *
* * * *
* * *
* *
*
```

Syntax

Step-1

	1	2	3	4	5	6	7	8	9
1					*				
2				*	*	*			
3			*	*	*	*	*		
4		*	*	*	*	*	*	*	
5	*	*	*	*	*	*	*	*	*

Step-2

i	j
1	5
2	4, 5, 6
3	3, 4, 5, 6, 7
4	2, 3, 4, 5, 6, 7, 8
5	1, 2, 3, 4, 5, 6, 7, 8

Step-3

i	j	and	j
1	$j \geq 5$		$j \leq 5$
2	$j \geq 4$		$j \leq 6$
3	$j \geq 3$		$j \leq 7$
4	$j \geq 2$		$j \leq 8$
5	$j \geq 1$		$j \leq 9$

Step-4

i	j	and	j
1	$j \geq 6-i$		$j \leq 4+i$
2	$j \geq 6-i$		$j \leq 4+i$
3	$j \geq 6-i$		$j \leq 4+i$
4	$j \geq 6-i$		$j \leq 4+i$
5	$j \geq 6-i$		$j \leq 4+i$

Step-5

$j \geq 6-i$ and $j \leq 4+i$

X - - - - Program - - - - X

```
for i in range(1,6):
    for j in range(1,10):
        if j >= 6-i and j <= 4+i:
            print("*", end=" ")
        else:
            print(" ", end=" ")
    print()
```

Output

```

      *
    * * *
  * * * * *
* * * * * * *
* * * * * * * *
```

X - - - - FormatSpecifier.py - - - - X

```
x=65
for i in range(1,6):
    for j in range(1,10):
        if j >= 6-i and j <= 4+i:
            print("%c" % x, end=" ")
        else:
            print(" ", end=" ")
    print()
```

print()

x=x+1

Output

```

      A
    B B B
  C C C C C
D D D D D D D
E E E E E E E E
```

{ %d = integer value }
 { %c = ~~character~~
 varchar value }

X - - - - - Format specifies - - - - - X

```
K=1
for i in range(1,6):
    for j in range(1,6):
        if j <= i:
            print(K, end=" ")
            K=K+1
        else:
            print(" ", end=" ")
    print()
    K=1
```

Output :-

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

X - - - - - Format specifies - - - - - X

```
K=1
for i in range(1,6):
    for j in range(1,6):
        if j <= i:
            print(K, end=" ")
        else:
            print(" ", end=" ")
    print()
    K=K+1
```

Output :-

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5
```

Program :-

```
for i in range(1,6):  
    for j in range(1,10):  
        if j <= 6-i or j >= 4+i:  
            print("*", end=" ")  
        else:  
            print(" ", end=" ")  
    print()
```

Output :-

```
* * * * * * * *  
* * * * * * *  
* * * * * *  
* * * * *  
* * * *  
* * *  
* *  
*  
* * * * *  
* * * * *  
* * * * *  
* * * * *  
* * * * *
```

Program :-

K=1

for i in range(1,6):

for j in range(1,10):

if $j \leq 6-i$ or $j \geq 4+i$:

print(k, end=" ")

else:

print(" ", end=" ")

print()

Output :-

```
1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1
1 1 1 1 1 1 1
1 1 1 1 1
1 1 1 1
1 1 1
1 1
1
```

X ————— X ————— X —————

Step-1 j

i =	1	2	3	4	5	6	7	8
1					1			
2				1	2	1		
3			1	2	3	2	1	
4		1	2	3	4	3	2	1
5	1	2	3	4	5	4	3	2

i = 1
1
2 1 2 1
3 1 2 3 2 1
4 1 2 3 4 3 2 1
5 1 2 3 4 5 4 3 2 1

Program

k = 1

```
for i in range(1, 6):  
    for j in range(1, 10):  
        if j >= 6 - i and j <= 4 + i:  
            print(k, end=" ")  
            if j < 5:  
                k = k + 1  
            else:  
                k = k - 1  
        else:  
            print(" ", end=" ")  
    print()  
    k = 1
```

Output

```
      1  
     1 2 1  
    1 2 3 2 1  
   1 2 3 4 3 2 1  
  1 2 3 4 5 4 3 2 1
```

Program :-

K = "*"

p = 1

for i in range(1,6):

for j in range(1,10):

if j >= 6-i and j <= 4+i and p == 1:

print(K, end=" ")

p = 0

else :

print(" ", end=" ")

p = 1

print()

Output

```
      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * * *
```

✧ Identity Operators ✧

In Python, Identity Operators are used to compare the memory locations of two objects to check whether they are exactly the same object in memory.

Identity Operators का उपयोग दो objects के Memory स्थानों की तुलना करने के लिए किया जाता है, ताकि यह जाना जा सके कि क्या वे Memory में बिल्कुल एक ही Object हैं।

OR

These operators used to check whether two objects are of same type or not and they share same memory location or not.

✧ Types of Identity Operators ✧

There are two types of Identity Operators.

1. `is`

2. `is not`

Note:- Always return "True or false" as a output.

Rules :-

1. **Used for memory comparison :-** यह Check करते हैं कि दो variables एक ही Memory स्थान को Point कर रहे हैं या नहीं।
2. **"is" Operator :-** यदि दोनों Variables एक ही Object को refer करते हैं, तो यह True, Return करेगा।
3. **"is not" Operator :-** यदि दोनों Variables अलग-अलग Objects को Refer करते हैं, तो यह True, Return करेगा।

4- Works with all data types :- You can use it with numbers, strings, lists, tuples etc.

5- "is" is not the same as "==" :- "Is", Object की पहचान (Identity) देखता है, जबकि "==" केवल Value की तुलना करता है।

6- Same values $\neq$ Same objects :- दो अलग-अलग Object की Value समान हो सकती है लेकिन उन्हें अलग-अलग Memory स्थानों में Store किया जा सकता है।

Ans:- दो List की Values same हो सकती हैं, लेकिन उनकी Memory अलग हो सकती है।

7- Immutable types (like integers, strings) may be cached :-

Python may reuse memory for small immutable values, so `a is b` might return true for some small integers.

इसलिए कभी-कभी छोटे नम्बरों या strings के लिए `is` True भी दे सकता है।

8. Should not be used to compare values :-

Is का प्रयोग केवल तब किया जाता है, जब Object identity जाननी हो (जैसे - None के साथ तुलना करते समय)

Ex -
 $x = 10$
 $y = 10$
 $\text{print}(x \text{ is } y)$
 $z = x$
 $\text{print}(z \text{ is } x)$

Output:-
 True
 True

$x = 10$
 $y = 10$
 $\text{print}(x \text{ is } y)$
 $z = x$
 $\text{print}(z \text{ is } x)$
 $\text{print}(\text{id}(x))$
 $\text{print}(\text{id}(y))$
 $\text{print}(\text{id}(z))$

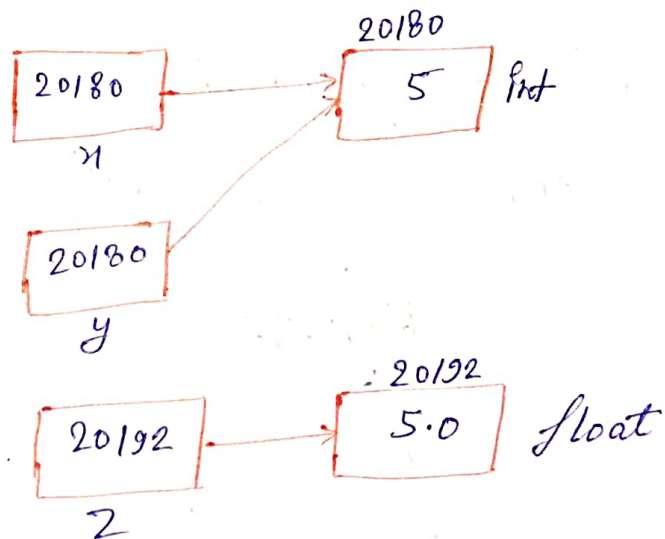
Output

True
 True
 140711626147016
 140711626147016
 140711626147016

$\text{list1} = [10, 20, 30]$
 $\text{list2} = [10, 20, 30]$
 $\text{print}(\text{list1 is list2})$
 $\text{print}(\text{id}(\text{list1}))$
 $\text{print}(\text{id}(\text{list2}))$

Output:-

False
 1853247395840
 1853204577344



Membership Operator

These operators used to test whether a specific value or item is present within a sequence, such as a string, list, tuple, or set.

OR

Membership operators are used to check if a value exists in a sequence like a list, string, tuple, set etc.

Membership Operator वो operator होते हैं जिनका उपयोग यह जाँचने (check) के लिए किया जाता है कि कोई value किसी sequence (जैसे - list, string, tuple, set etc.) में मौजूद है या नहीं।

→ There are two membership operators: "in" & "not in"

→ Always return "True or False" as a output.

Key Points

~~1. Used to check presence :-~~

1. Two main operators :-

→ in :- यदि Value मौजूद है, तो True।

→ not in :- यदि Value मौजूद नहीं है, तो True

2. Works with different sequences :-

These operators can be used with:

→ String ("a" in "apple")

→ List (5 in [1, 2, 5])

→ Tuple (3 in (1, 2, 3))

→ Set ('x' in {'x', 'y'})

→ Dictionary (key in {'a': 1}) → [Checks key only]

3. Case-sensitive in strings:-

"a" in "Apple" → False (क्योंकि 'A' और 'a' अलग होते हैं)

Note:-

Because of case difference $A \neq a$

4. Works only iterable objects:-

in या not in के बाद जो object आता है, वो list, string, tuple जैसे iterable होना चाहिए।

5. Used in loops and conditions:-

Commonly used in if statements and for loops.

As:- if "apple" in fruits: या for item in list:

Ex.

```
reg_user = ["aryan", "amit", "ankit", "laxmi"]
```

```
name = input("Enter your name: ")
```

```
if name in reg_user:
```

```
    print("Access granted. Welcome bro")
```

```
else:
```

```
    print("Access denied. You are not registered.")
```

Output :-

Enter your name: aryan

Access granted. Welcome bro.

Escape Characters

There are special characters used to represent non-printable or special character with a string.

As :-

Escape Code	Meaning	Example	Output
\n	new line	print("Hello\nWorld")	Hello World
\t	Tab space	print("Hello\tWorld")	Hello World
\'	Single quote	print('It\'s fine')	It's fine
\"	Double quote	print("He said \"Hi\"")	He said "Hi"
\\	Backslash	print("C:\\folder\\file")	C:\folder\file

Arithmetic Operators

These operators are used to perform basic arithmetic operations such as addition, subtraction, multiplication, division, modulus and exponentiation.

Arithmetic Operators का use हम Mathematics के सामान्य काम जैसे जोड़, घटाव, गुणा और भाग आदि करने के लिए करते हैं।

As :-

Operators	Name	Example	Output
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
//	Floor Division	5 // 2	2
%	Modulus (Remainder)	5 % 2	1
**	Exponent (Power)	5 ** 2 $\rightarrow$ 5 ²	25

Logical Operator

Logical Operators are used to check multiple conditions at once.

Note:- There are three logical operators in python.

1. **and** :- It returns True if both the operands are True, otherwise it returns false.

जब दोनों Conditions, True हों तब ही Result, True होगा
Otherwise कोई भी एक गलत हो, तो Result, False होगा।

Ex:
True and True $\rightarrow$ True
True and False $\rightarrow$ False
False and False $\rightarrow$ False

2. **or** :- It returns true if at least one of the operands is True, otherwise it returns false.

यदि कोई भी एक Condition सही हो, तो Result, True होगा
है otherwise दोनों Condition false हों, तो Result False होगा।

Ex.
True or False $\rightarrow$ True
False or False $\rightarrow$ False
True or True $\rightarrow$ True

3. **Not** :- It returns ~~True~~ the opposite Boolean value of the operands ~~if the operand is True, it returns false~~ and if the operand is false, it returns True.

यह किसी भी Condition का उल्टा (opposite) कर देगा है।
True को False और False को True बना देगा है।

Ex.
not True $\rightarrow$ False
not False $\rightarrow$ True.

Tables

True	and	True	True
True	and	False	False
False	and	False	False
True	or	False	True
False	or	False	False
True	or	True	True

Comparison Operators

In Python, comparison operators are used to compare two values and return a Boolean value (True or False) depending on the result of the comparison.

1. **==(Equal to)** :- Returns True if the values on either side of the operator are equal and False otherwise.

True तब मिलेगा जब दोनों मान बराबर हों, यदि दोनों में कोई फर्क हो, तो False मिलेगा।

Ex.

5 == 5 # True

5 == 3 # False

2. **!=(Not equal to)** :- Returns True if the values on either side of the operators are not equal and False otherwise.

True तब मिलेगा जब दोनों मान बराबर नहीं हों यदि दोनों सामान हों तो False मिलेगा।

Ex.

5 != 3 # True

5 != 5 # False

3. $>$ (Greater than) :- Returns True if the value on the left is greater than the value on the right and false otherwise.

↳ True तब मिलेगा जब Left Side वाला मान Right Side से बड़ा हो
↳ यदि Left Side वाला मान छोटा या बराबर हो, तो False मिलेगा।

Ex.
 $5 > 3$ # True
 $5 > 8$ # False

4. $<$ (Less than) :- Returns True if the value on the left is less than the value on the right and false otherwise.

↳ जब बाएं वाला मान दाएं से छोटा हो तो True मिलेगा।
↳ यदि बाएं वाला मान बड़ा या बराबर हो, तो False मिलेगा।

Ex.
 $3 < 5$ # True
 $8 < 5$ # False

5. $>=$ (Greater than or equal to) :- Return True if the value on the left is greater than or equal to the value on the right, and false otherwise.

↳ True तब मिलेगा जब बाएं वाला मान दाएं से बड़ा या बराबर हो।
↳ अगर बाएं वाला मान छोटा हो, तो False मिलेगा।

Ex.
 $5 >= 5$ # True
 $3 >= 5$ # False

6- $\leq$ (Less than or equal to) :- Return True if the value on the left is less than or equal to the value on the right and false otherwise.

↳ जब बाएं वाला मान दाएं से छोटा या बराबर हो ।
↳ यदि बाएं वाला मान बड़ा हो, तो False मिलेगा ।

Ex.

$3 \leq 5$ # True

$5 \leq 3$ # False

OPERATOR	Meaning	Example	Result.
$==$	Equal to	$5 == 5$	True
$!=$	Not Equal to	$5 != 3$	True
$>$	Greater than	$5 > 3$	True
$<$	Less than	$5 < 3$	False
$>=$	Greater than or equal to	$5 >= 5$	True
$\leq$	Less than or equal to	$3 \leq 5$	True

Program of List

x = [10, 20, 30]

x.append(40)

print(x)

↳ [10, 20, 30, 40]

x.insert(1, 15)

print(x)

↳ [10, 15, 20, 30, 40]

x.reverse()

print(x)

↳ 40, 30, 20, 15, 10

x = [10, 20, 30, 40, 50, 60]

y = x.copy()

z = x

y

[10, 20, 30, 40, 50, 60]

z

[10, 20, 30, 40, 50, 60]

x

[10, 20, 30, 40, 50, 60]

Slice Operation

In Python, the slice operator is used to extract a portion (part) of a sequence like a list, string, or tuple.

Slice operation का use किसी sequence (जैसे- list, string, tuple) से कुछ हिस्से (part) को निकालने के लिए किया जाता है।

Syntax :-

`sequence[start : stop : step]`

Note :-

- i- start :- कहाँ से slicing शुरू होनी है (Index शामिल होगा)
- ii- stop :- कहाँ तक slicing करनी है (Index शामिल नहीं होगा)
- iii- step :- कितनी-कितनी jump में values लेनी हैं।

Rules of Slice Operator :-

1. Default values :-

- start की default value = 0
- stop की default value = len(sequence)
- step की default value = 1

2. Stop index exclusive होता है, यानी वो Index शामिल नहीं होता है।

Ex. `a = [10, 20, 30, 40]`

`print(a[0:2])` # (index 2 नहीं आएगा)

Output :-

`[10, 20]`

3. यदि Step Positive है, तो Slicing Left to Right होगी।
4. यदि Step Negative है, तो Slicing Right to Left होगा।

Ex. $x = [1, 2, 3, 4]$
 $\text{print}(x[::-1])$

Output:-

$[4, 3, 2, 1]$

5. यदि Start या Stop को skip कर दो तो Default value use होगी।

Ex. $x = [10, 20, 30, 40, 50]$

$\text{print}(x[:3])$ # Output: $[10, 20, 30]$

$\text{print}(x[2:])$ # Output: $[30, 40, 50]$

$\text{print}(x[:])$ # Output: $[10, 20, 30, 40, 50]$

6. Negative indexing भी चलती है।

Ex. $x = [10, 20, 30, 40, 50]$

$\text{print}(x[-3:-1])$

Output:

$[30, 40]$

7. यदि Step, 0 रखा, तो Exception आरगा।

Ex. $x = [1, 2, 3]$

$\text{print}(x[::0])$ # Exception: slice step cannot be zero

X — — — Slice Program — — — X

$\mathcal{A}_1 = [70, 50, 80, 9, 12, 25, 91, 86]$

$\mathcal{A}_1[2:5:]$ # Output: $[80, 9, 12]$

$\mathcal{A}_1[:4:]$ # Output: $[70, 50, 80, 9]$

$\mathcal{A}_1[2::]$ # Output: $[80, 9, 12, 25, 91, 86]$

$\mathcal{A}_1[::2]$ # Output: $[70, 80, 12, 91]$

$\mathcal{A}_1[-1:0:-1]$ # Output: $[86, 91, 25, 12, 9, 80, 50]$

$\mathcal{A}_1[-1::-1]$ # Output: $[86, 91, 25, 12, 9, 80, 50, 70]$